Category: Model Card • **Reading time:** 20 minutes 17 August 2026

PatchFM: A foundation model for Univariate Time Series Forecasting

A concise and reproducible recipe for training a Transformer Decoder-based forecasting model for univariate time series. The approach mirrors LLM practices (next-token ⟶ next-patch) while remaining lightweight compared to a classic LLM and practical.

Samy-Melwan Vilhes

samy-melwan.vilhes@insa-rouen.fr
INSA Rouen Normandie

Abstract

In recent years, the success of large language models (LLMs) in text generation has inspired analogous approaches in time series forecasting. Large-scale models have been designed to take a signal as input and generate its future values. Rather than being limited to one signal type, these models are trained to generalize across diverse and even unseen signals, a property referred to as zero-shot forecasting. They can be pretrained using Transformer Decoder-based architectures and training strategies similar to those employed in LLMs. In this work, we propose a recipe for training such a model, inspired by the paradigm of LLMs. While LLMs rely on next-token prediction, our model is specifically designed for next-patch prediction in time series. Our goal is not to propose an entirely new approach, but to make both the model and its pretraining process transparent, accessible, and reproducible, while avoiding the need for massive computational resources (training on a single H100 GPU for 20 hours). By drawing on established methods and thoughtfully combining their components, we deliver a model and pretraining process that are both practical and easy to reproduce.

Keywords: Time Series Forecasting, Foundation Models, Transformer Decoders

Github: https://github.com/vilhess/PatchFM

1. Introduction

1.1. Related Work

Motivated by the success of Large Language Models (LLMs) [1], [2], large time-series models aim to learn transferable representations from large-scale and heterogeneous corpora of signals. To this end, architectures have been developed for time-series forecasting and typically adopt patch-based representations [3], the equivalent of tokens in LLMs.

However, the continuous nature of time-series data introduces additional challenges. The statistical heterogeneity of real-world signals — manifested in varying means, variances, and distributional shifts — makes it particularly difficult to train models that generalize across large and diverse corpora. Unlike LLMs, which operate on discrete token vocabularies with a classification head, state-of-the-art time-series foundation models rely on a regression head to predict the next patch of continuous values, a design choice directly motivated by the continuous nature of the data.

Moirai-2 [4] and TimesFM [5] are examples of Transformer-decoder-based large time-series models trained using patching, primarily focusing on univariate forecasting. TiRex [6] follows a similar approach but replaces the Transformer architecture

with xLSTMs [7]. Toto [8] and Chronos-2 [9] are also Transformer-based large time-series models trained with patching; however, they additionally address multivariate forecasting through a cross-variate attention mechanism. Toto [8] and Toto-2 [10] adopt decoder-based architectures, while Chronos-2 [9] adopts an encoder design. Other approaches adopt time-step-based rather than patch-based representations, such as StateFlow [11], which relies on state-space models [12], and TimeMoE [13], which uses a Transformer-based mixture-of-experts architecture. These models are likewise trained using an efficient causal strategy.

1.2. The hidden training recipe

As in the LLM literature, the training recipes of state-of-the-art time-series foundation models are often only partially disclosed: while model architectures and high-level training procedures are described in publications, the complete training data and code are typically not publicly available. In this model card, we unveil the training recipe of PatchFM, a Transformer decoder-based foundation model for univariate time-series forecasting. We provide a concise and reproducible guide (with the code) for training such a model by leveraging established architecture components, training strategies, publicly available datasets, and synthetic data. By carefully combining these components, we present a practical and accessible framework for developing competitive time-series foundation models.

2. Model Architecture

2.1. Patch-based representations

Patching for time-series was introduced in [3], where a patch-based Transformer architecture was proposed for forecasting. This paradigm has since become a standard component of large time-series models. Patches correspond to contiguous segments of a time-series and play a role analogous to tokens in large language models.

Formally, an input signal $\mathbf{x} \in \mathbb{R}^{\mathrm{w}}$ of length w is divided into consecutive patches of length L. This reduces the context length by a factor of L, significantly lowering the computational cost of training and inference. It is the 1-dimensional equivalent of the image patching used in vision transformers [14].

Let $\mathrm{N} = \lfloor \frac{\mathrm{w}}{\mathrm{L}} \rfloor$ be the number of patches and

$$\mathbf{x}_{\mathrm{P}} = \left(\mathbf{x}_{\mathrm{P}_1}, \cdots, \mathbf{x}_{\mathrm{P}_{\mathrm{N}}}\right)^{\top} \in \mathbb{R}^{\mathrm{N}x\mathrm{L}}$$

where the i-th patch is denoted by $\mathbf{x}_{\mathrm{P}_i}$. Models are trained to predict the next patch $\mathbf{x}_{\mathrm{P}_{\{i+1\}}}$ given the previous patches $\mathbf{x}_{\mathrm{P}_1}, ..., \mathbf{x}_{\mathrm{P}_i}$ for $i = 1, ..., \mathrm{N} - 1$. With this strategy, a single forecast estimates the next patch, so the L future observations are predicted at once, which is more efficient than forecasting one observation at a time.

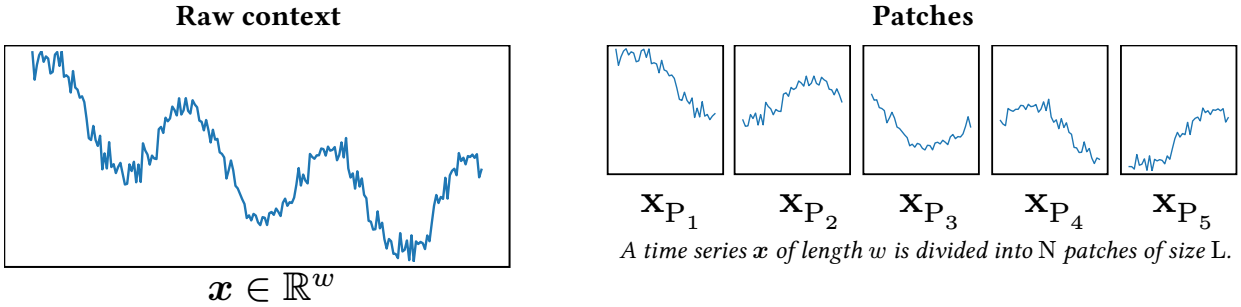
2.2. Normalization Strategy

Unlike discrete tokens, time-series are continuous-valued, which necessitates normalization to ensure robustness across signals with varying scales. Given a patch $\mathbf{x}_{\mathrm{P}_i}$, normalization for a given patch is defined as:

$$\tilde{\mathbf{x}}_{\mathrm{P}_i} = \frac{\mathbf{x}_{\mathrm{P}_i} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}},$$

where μ_i and σ_i^2 are the mean and variance used to normalize $\mathbf{x}_{\mathrm{P}_i}$ and computed by a given *normalization strategy*. Here, the parameter $\varepsilon > 0$ ensures numerical stability.

Preserving the causal nature of the Transformer decoder requires that normalization statistics be computed solely from past observations, without leaking future information [15]. In PatchFM, we adopt a causal normalization strategy augmented with the $\sinh^{-1}$ transformation, following [9] and [10]. The



A time series $\boldsymbol{x}$ of length w is divided into N *patches of size* L.

running mean and variance are estimated over the causally available context, and the $\sinh^{-1}$ function is subsequently applied to the normalized signal to robustly handle heavy-tailed distributions and outliers while maintaining causality.

The normalization statistics for the i-th patch are computed as follows:

$$\mu_i = \text{Mean}\left(\mathbf{x}_{\text{P}_1}, ..., \mathbf{x}_{\text{P}_i}\right), \sigma_i = \text{Std}\left(\mathbf{x}_{\text{P}_1}, ..., \mathbf{x}_{\text{P}_i}\right).$$

Combining the causal normalization with the $\sinh^{-1}$ transformation, the final normalized patch fed to the Transformer decoder is:

$$\tilde{\mathbf{x}}_{\text{P}_i} = \sinh^{-1}\left(\frac{\mathbf{x}_{\text{P}_i} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}\right).$$

2.3. Transformer Decoder-based architecture

PatchFM is built on the Transformer Decoder [16], the architecture underlying most modern LLMs. We employ pre-normalization, causal masked multi-head attention, and a SiLU Gated Feed-Forward Network, with Rotary Positional Encoding [17] to model temporal dependencies. A key advantage of this architecture is training efficiency: given a sequence of N patches, the model predicts all $\text{N} - 1$ successive patches in a single forward pass, computing the loss over all predictions simultaneously rather than one patch at a time.

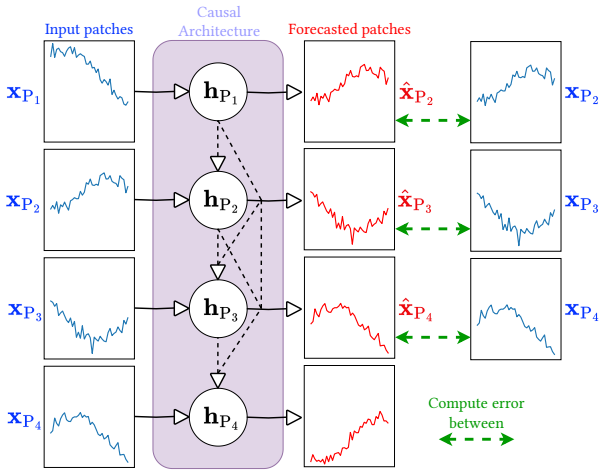


Figure 3: Causal training strategy with patch-based representations.

Before entering the Transformer decoder, each normalized patch is projected into a higher-dimensional embedding space via a Feed-Forward patch embedding layer. The decoder output is then passed to a quantile regression head (a Feed-Forward Network) that predicts a set of quantiles of the next patch distribution: the median serves as the point forecast, while the remaining quantiles provide calibrated uncertainty estimates.

Causal normalization combined with the Transformer decoder further enables key-value (KV) caching [18] at inference time, avoiding redundant context recomputations and substantially reducing the cost of long-horizon autoregressive forecasting.

2.4. Denormalization

Since the model operates in the normalized space, its quantile predictions must be mapped back to the original scale. Given the predicted quantile $\hat{\mathbf{x}}^{(q)}_{\text{P}_{i+1}}$ and the normalization statistics μ_i and σ_i from the context, the denormalized forecast is:

$$\hat{\mathbf{x}}^{(q)}_{\text{P}_{i+1},\ \text{denorm}} = \sinh\left(\hat{\mathbf{x}}^{(q)}_{\text{P}_{i+1}}\right) \cdot \sqrt{\sigma_i^2 + \varepsilon} + \mu_i.$$

2.5. Hyperparameters (evolve over time)

The model operates on a context of 1024 observations with a patch size of 32, yielding a sequence of 32 patches fed to the decoder. The Transformer decoder has 6 layers, a model dimension of 2048, and 32 attention heads (head dimension 64). The quantile regression head outputs 9 quantiles: $\mathcal{Q} = \{0.1, 0.2, ..., 0.9\}$ per time step in a patch. The model totals approximately 350 million parameters, lightweight relative to typical LLMs, but large by time-series forecasting standards.

3. Training Recipe

3.1. Datasets

PatchFM is pretrained on a diverse mixture of real-world and synthetic univariate time series, spanning multiple domains and sampling frequencies. The training corpus comprises five components.

GIFT-Pretraining [19]. The GIFT-Pretraining split provides approximately 600K univariate series drawn from 71 univariate and 17 multivariate datasets across diverse domains and frequencies. This split is explicitly curated to avoid data leakage with respect to the GIFT-Eval benchmark.

Chronos synthetic data [20]. Two large-scale synthetic datasets are generated following the Chronos methodology: a TSMixup dataset of approximately 10 million univariate series and a KernelSynth dataset of approximately 1 million series, each of length 1024.

Artificial signals. Approximately 1 million deterministic synthetic series (sinusoidal, linear, polynomial, and logarithmic functions), further augmented with signal mixtures via TSMixup [20] and Gaussian Process samples from KernelSynth (combinations of RBF, periodic, and linear kernels with swept hyperparameters).

BOOM [8]. Approximately 5 million series of length 1024 sampled from the BOOM dataset. We note that BOOM is also used as an evaluation benchmark; both PatchFM variants (with and without benchmark leakage) include BOOM in their training data to increase corpus diversity. Consequently, neither variant can be fairly evaluated on the BOOM benchmark. An alternative checkpoint trained without any BOOM data is available upon request.

TSMixup augmentation [20]. Additional series are generated by applying TSMixup to the synthetic and GIFT-Pretraining datasets, further increasing diversity through signal interpolation.

We additionally release a variant trained with the full GIFT-Eval split included, providing broader coverage of real-world signals at the cost of fair evaluation on the GIFT-Eval benchmark. This leakage-aware checkpoint is intended for production use, where benchmark integrity is not a concern.

Each training signal is randomly flipped ($x \mapsto -x$) and temporally reversed ($x \mapsto x[::-1]$), each independently with probability 0.5, to further increase corpus diversity.

3.2. Multi-quantile (Pinball) Loss

The model jointly forecasts the next patch and estimates predictive uncertainty [4], [6], [9], [10]. It outputs a fixed quantile set $\mathcal{Q} = \{0.1, 0.2, ..., 0.9\}$, where the median ($q = 0.5$) serves as the point forecast and the remaining quantiles provide uncertainty bands.

For each position $i \in \{1, ..., \mathrm{N}\}$, the model predicts quantiles of the next patch $\mathbf{x}_{\mathrm{P}_{i+1}}$ conditioned on $\mathbf{x}_{\mathrm{P}_1}, ..., \mathbf{x}_{\mathrm{P}_i}$. We denote the prediction at quantile q by $\hat{\mathbf{x}}^{(q)}_{\mathrm{P}_{i+1}} \in \mathbb{R}^{\mathrm{L}}$.

The training objective is the multi-quantile pinball loss, aggregated over positions, patch elements, and quantiles:

$$\mathcal{L} = \frac{1}{\mathrm{NL}\ |\mathcal{Q}|} \sum_{i=1}^{\mathrm{N}} \sum_{t=1}^{\mathrm{L}} \sum_{q \in \mathcal{Q}} \rho_q \Big(\tilde{\mathbf{x}}_{\mathrm{P}_{i+1},t} - \hat{\mathbf{x}}^{(q)}_{\mathrm{P}_{i+1},t} \Big),$$

where the pinball loss ρ_q is defined as:

$$\rho_q(u) = \begin{cases} qu & \text{if } u \geq 0 \\ (q-1)u & \text{if } u < 0. \end{cases}$$

Importantly, the loss is computed in the normalized space: the target patch is transformed as $\tilde{\mathbf{x}}_{\mathrm{P}_{i+1}} = \sinh^{-1}\left(\frac{\mathbf{x}_{\mathrm{P}_{i+1}} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}\right)$, where μ_i and σ_i are the causal statistics of patches $\mathbf{x}_{\mathrm{P}_1}, ..., \mathbf{x}_{\mathrm{P}_i}$, and compared directly against the model outputs $\hat{\mathbf{x}}^{(q)}_{\mathrm{P}_{i+1}}$. Since the normalization is strictly increasing, quantiles are equivariant under it: the q-quantile in the normalized space maps back to the q-quantile of the original series. The denormalization step is therefore applied only at inference time. Computing the loss in this space bounds the per-element error regardless of the target's magnitude relative to the causal history, which prevents the gradient amplification that the sinh denormalization would otherwise introduce during training.

3.3. Training Details

Training was performed on a single NVIDIA H100 GPU using PyTorch Lightning and a global batch size of 512.

Following [10], we optimize the model with Muon [21] instead of only AdamW. Muon updates each weight matrix by orthogonalizing its momentum buffer through a Newton-Schulz iteration before applying the step, which equalizes the contribution of every singular direction of the update and has been shown to improve token efficiency at a lower optimizer memory cost. Muon acts on the two-dimensional hidden-layer parameters of the decoder; all scalar parameters (normalization gains and biases) are optimized with AdamW, as prescribed in [21].

A practical difficulty of this hybrid setup is that the raw orthogonalized update and the AdamW update do not share the same magnitude, so the two para-

meter groups cannot use a common learning rate. We therefore adopt the implementation of [22], which rescales the update of a weight matrix $\mathbf{W} \in \mathbb{R}^{n\times m}$ as

$$\mathbf{W} \leftarrow \mathbf{W} - \eta\Big(0.2 \cdot \sqrt{\max(n,m)} \cdot \mathrm{NewtonSchulz}(\mathbf{M}) + \lambda \mathbf{W}\Big),$$

where $\mathbf{M}$ is the momentum buffer and λ the decoupled weight-decay coefficient. The factor $0.2\sqrt{\max(n,m)}$ matches the root-mean-square magnitude of the Muon update to that of an AdamW update, so both parameter groups can share a single learning rate and schedule, and the hyperparameters tuned for AdamW transfer directly without retuning.

The learning rate schedule consists of a linear warm-up from 10^{-5} to 5×10^{-4} over 10k steps, followed by cosine decay back to 10^{-5} until 150k steps, and a constant rate thereafter. As indicated, two variants of PatchFM are trained: one with GIFT-Eval data leakage and one without.

4. Inference and Evaluation

4.1. Autoregressive Quantile Decoding

During autoregressive inference, the model generates forecasts patch by patch. At each step, the predicted patch is appended to the context and fed back as input for the next step, iterating until the desired horizon is reached.

Quantile forecasting introduces additional complexity: rather than outputting a single patch per step, the model produces $|\mathcal{Q}|$ patches, one per quantile. Since the autoregressive step requires a single patch as input, it is not straightforward to feed all quantile predictions back simultaneously.

A workaround used in [4] is to use only the median prediction ($q = 0.5$) as the autoregressive input. While this preserves the autoregressive structure, it discards the uncertainty captured by the remaining quantiles.

An alternative is autoregressive multi-quantile decoding [4], which maintains full predictive uncertainty throughout generation. The key idea is to maintain $|\mathcal{Q}|$ parallel contexts — one per quantile path — and at each step aggregate across all paths to obtain calibrated quantile estimates. This incurs a batch-size factor of $|\mathcal{Q}|$ but remains tractable in practice.

The algorithm proceeds as follows:

1. **Initialization.** Start with the initial context of observed data. Notation: `BS` batch size; `L` context length; `P` patch size; `Q` number of quantiles; `H` forecast horizon. **Context shape:** `(BS × L)`.
2. **First quantile prediction.** Predict quantiles for the next patch from the current context. **Output shape:** `(BS × P × Q)`.
3. **Context duplication.** For each predicted quantile, append the corresponding patch to the context, creating `Q` separate branches. **New context shape:** `(BS × Q × (L + P))`.
4. **Next forward pass.** For each branch, predict quantiles of the following patch. **Output shape:** `(BS × Q × P × Q)`.
5. **Quantile collapse.** Reshape to `(BS × P × Q²)` and compute quantiles across all `Q²` paths to obtain calibrated estimates for the next patch. **Output shape:** `(BS × P × Q)`. Increment the step counter.
6. **Iteration.** Repeat steps 3-5 until `steps × P ≥ H`.

4.2. Flip equivariance

At inference time, the model performs forecasts on both the original context $\mathbf{x}$ and its amplitude-flipped counterpart $-\mathbf{x}$ in a single batched forward pass. The final forecast is the odd part of the model output:

$$\hat{\mathbf{x}} = \frac{f(\mathbf{x}) - f(-\mathbf{x})}{2}.$$

Since the model is trained with random amplitude flipping, it approximately satisfies $f(-\mathbf{x}) \approx -f(\mathbf{x})$, so this combination cancels any residual even (non-equivariant) component and retains only the equivariant part of the predictions. This approach, first applied in [23], improves robustness at the cost of doubling the batch size.

4.3. Evaluation

Evaluation is performed on FEV-Bench [24], a benchmark of 100 forecasting tasks across seven domains. Since FEV-Bench includes subsets of GIFT-Eval data, we report results for the variant trained without GIFT-Eval leakage to ensure fair comparison. The leakage-aware variant is evaluated on TS-Arena [25], a live forecasting platform where models must submit predictions before the ground-truth data exists,

making test-set contamination impossible by design. TS-Arena is currently composed primarily of energy-related forecasting tasks.

We do not re-run baseline experiments; instead, we use results from the respective benchmark repositories. For FEV-Bench, results are taken from the official repository; for TS-Arena, we use the live leaderboard.

4.3.1. FEV-Bench

PatchFM (without GIFT-Eval leakage) ranks 12th out of 24 entries on FEV-Bench in both win rate and skill score (relative to Seasonal Naive as baseline). Despite its modest training budget, it matches the previous generation of state-of-the-art foundation models: it obtains exactly the same win rate and skill score as Toto-1.0 [8], and outperforms Moirai-2.0 [4] and the smallest Toto-2.0 variant [10]. The current leaders — Chronos-2 [9], TiRex-2 [6], and the larger Toto-2.0 checkpoints [10] — remain ahead, which is expected given that they are trained on substantially larger corpora and compute budgets.

Model	Win Rate	Skill Score
Chronos-2	0.81	0.47
TiRex-2	0.78	0.45
Toto-2.0-2.5B	0.78	0.44
Toto-2.0-1B	0.77	0.44
Toto-2.0-313m	0.75	0.44
TimesFM-2.5	0.71	0.47
TiRex	0.70	0.43
Toto-2.0-22m	0.67	0.43
TS-ICL	0.61	0.43
FlowState	0.60	0.42
TabPFN-TS-3	0.60	0.43
PatchFM	**0.58**	**0.41**
citras-fm	0.58	0.41
Toto-1.0	0.58	0.41
Toto-2.0-4m	0.55	0.41
Moirai-2.0	0.53	0.40
Chronos-Bolt	0.49	0.39
TFT	0.37	0.32
Sundial-Base	0.32	0.34
PatchTST	0.31	0.30
CatBoost	0.22	0.23
LightGBM	0.21	0.21
AutoTheta	0.20	0.05
Seasonal Naive	0.13	0.00

Table 1: FEV-Bench results, sorted by win rate. Win rate and skill score are computed relative to Seasonal Naive. PatchFM is the variant trained without GIFT-Eval leakage.

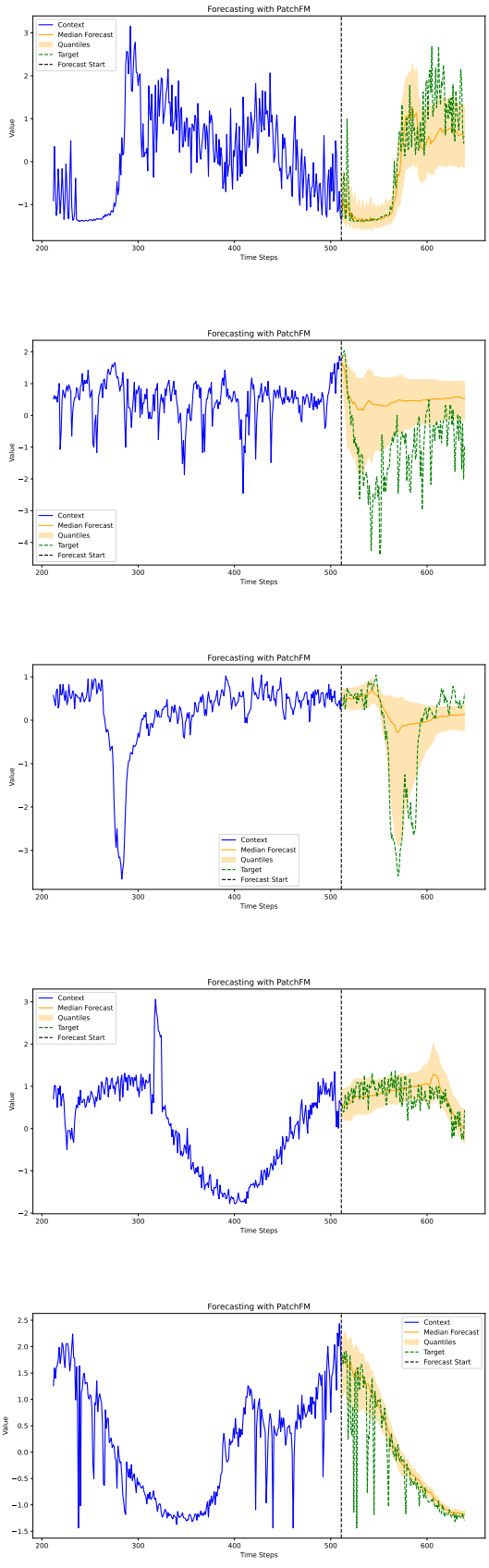
4.3.2. TS-Arena

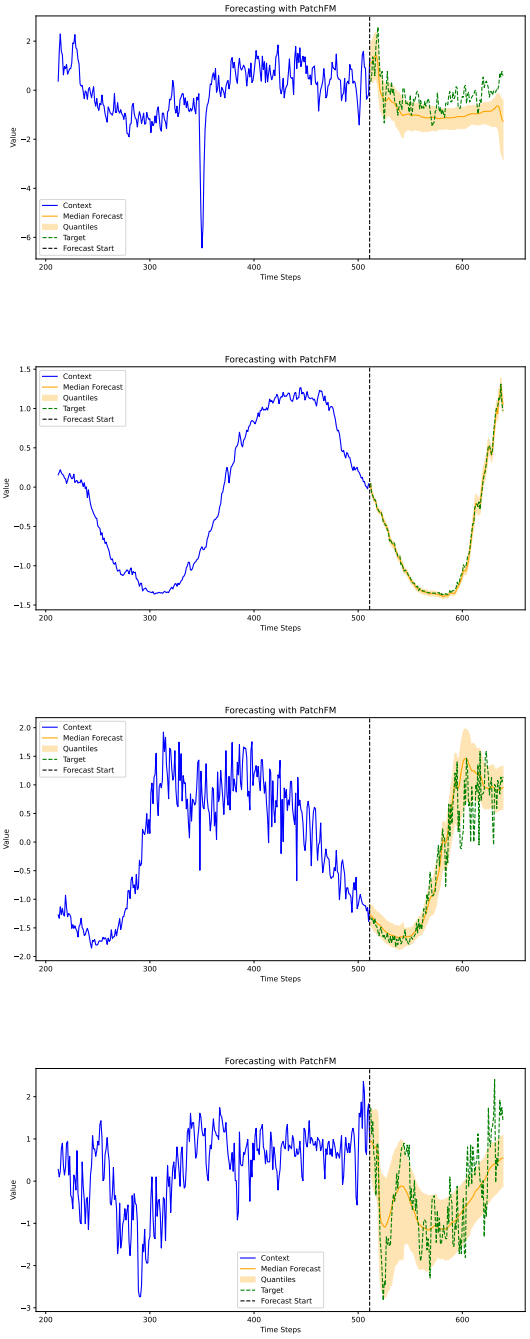
TS-Arena results are available live at the leaderboard (https://ts-arena.live). PatchFM (with leakage), submitted as "LITIS/PatchFM-Large", holds a 4th-5th position ELO score, competing closely with Moirai-2.0 [4]. The leaderboard is currently dominated by Chronos-2 [9] and TiRex [6].

4.3.3. Visualisation of forecasts on GIFT-Eval

Below are visualizations of PatchFM (without GIFT-Eval leakage) forecasts on GIFT-Eval test signals, showcasing the model's ability to capture complex temporal patterns and provide calibrated uncertainty

estimates. The shaded areas represent the predicted quantile intervals, while the solid line corresponds to the median forecast.





5. Open Source Contribution

The code, training data curation scripts, and model checkpoints (huggingface) are publicly available at github.com/vilhess/PatchFM. We welcome contributions from the community, whether to improve the model architecture, extend the training recipe, or push performance closer to state-of-the-art, with the shared goal of making competitive time-series foundation models more accessible and reproducible.

Bibliography

[1] OpenAI, :, S. Agarwal, L. Ahmad, J. Ai, and S. A. et al, "gpt-oss-120b and gpt-oss-20b Model Card." [Online]. Available: https://arxiv.org/abs/2508.10925

[2] A. Grattafiori, A. Dubey, A. Jauhri, and A. P. et al, "The Llama 3 Herd of Models." [Online]. Available: https://arxiv.org/abs/2407.21783

[3] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=Jbdc0vTOcol

[4] C. Liu *et al.*, "Moirai 2.0: When Less Is More for Time Series Forecasting." [Online]. Available: http://arxiv.org/abs/2511.11698

[5] A. Das, W. Kong, R. Sen, and Y. Zhou, "A decoder-only foundation model for time-series forecasting." [Online]. Available: https://arxiv.org/abs/2310.10688

[6] A. Auer, P. Podest, D. Klotz, S. Böck, G. Klambauer, and S. Hochreiter, "TiRex: Zero-Shot Forecasting Across Long and Short Horizons with Enhanced In-Context Learning." [Online]. Available: https://arxiv.org/abs/2505.23719

[7] M. Beck *et al.*, "xLSTM: Extended Long Short-Term Memory." [Online]. Available: https://arxiv.org/abs/2405.04517

[8] B. Cohen *et al.*, "This Time is Different: An Observability Perspective on Time Series Foundation Models." [Online]. Available: https://arxiv.org/abs/2505.14766

[9] A. F. Ansari *et al.*, "Chronos-2: From Univariate to Universal Forecasting." [Online]. Available: https://arxiv.org/abs/2510.15821

[10] E. Khwaja *et al.*, "Toto 2.0: Time Series Forecasting Enters the Scaling Era." [Online]. Available: https://arxiv.org/abs/2605.20119

[11] L. Graf, T. Ortner, S. Woźniak, and A. Pantazi, "FlowState: Sampling Rate Invariant Time Series Forecasting." [Online]. Available: https://arxiv.org/abs/2508.05287

[12] J. T. H. Smith, A. Warrington, and S. W. Linderman, "Simplified State Space Layers for Sequence Modeling." [Online]. Available: https://arxiv.org/abs/2208.04933

[13] X. Shi *et al.*, "Time-MoE: Billion-Scale Time Series Foundation Models with Mixture of Experts." [Online]. Available: https://arxiv.org/abs/2409.16040

[14] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.

[15] S.-M. Vilhes, G. Gasso, and M. Z. Alaya, "Does Normalization Choice Matter for Causal Large Time-Series Models?," in *1st ICLR Workshop on Time Series in the Age of Large Models*, 2026. [Online]. Available: https://openreview.net/forum?id=lMNWBnFHxt

[16] A. Vaswani *et al.*, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, in NIPS'17. Long Beach, California, USA: Curran Associates Inc., 2017.

[17] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding." [Online]. Available: https://arxiv.org/abs/2104.09864

[18] W. Gao, X. Zhou, P. Sun, T. Zhang, and Y. Wen, "Rethinking Key-Value Cache Compression Techniques for Large Language Model Serving." [Online]. Available: https://arxiv.org/abs/2503.24000

[19] T. Aksu *et al.*, "GIFT-Eval: A Benchmark For General Time Series Forecasting Model Evaluation." [Online]. Available: https://arxiv.org/abs/2410.10393

[20] A. F. Ansari *et al.*, "Chronos: Learning the Language of Time Series." [Online]. Available: https://arxiv.org/abs/2403.07815

[21] K. Jordan *et al.*, "Muon: An optimizer for hidden layers in neural networks." [Online]. Available: https://kellerjordan.github.io/posts/muon/

[22] J. Liu *et al.*, "Muon is Scalable for LLM Training." [Online]. Available: https://arxiv.org/abs/2502.16982

[23] X. Fu, Y. Li, G. Papaioannou, and Y. Kim, "Reverso: Efficient Time Series Foundation Models for Zero-shot Forecasting." [Online]. Available: https://arxiv.org/abs/2602.17634

[24] O. Shchur *et al.*, "fev-bench: A Realistic Benchmark for Time Series Forecasting." [Online]. Available: https://arxiv.org/abs/2509.26468

[25] M. Meyer, S. Kaltenpoth, H. Albers, K. Zalipski, and O. Müller, "TS-Arena – A Live Forecast Pre-Registration Platform." [Online]. Available: https://arxiv.org/abs/2512.20761